



GESP三级直通车

一篇梳理GESP编程能力等级考试三级C++语言全部知识体系，吃透难点，摆平易错点，掌握高频考点。



GESP三级考纲知识点

知识点

考察频率

数据编码（原码、反码、补码）	多
进制转换（二进制、八进制、十进制、十六进制）	很多
位运算（&， ，~，^,<<,>>）	很多
字符串及其函数	一般
一维数组基本应用	多
模拟法	编程题必考
枚举法	编程题必考

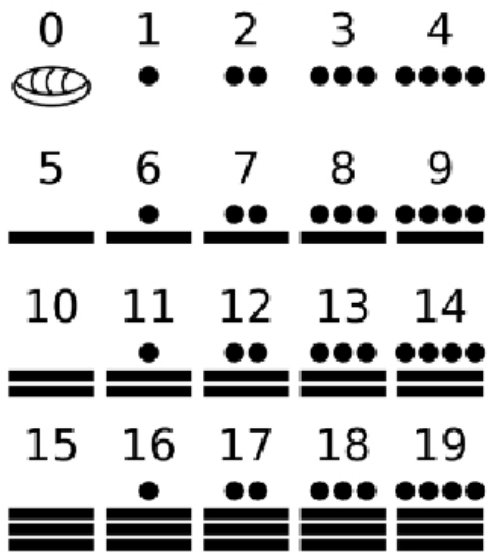
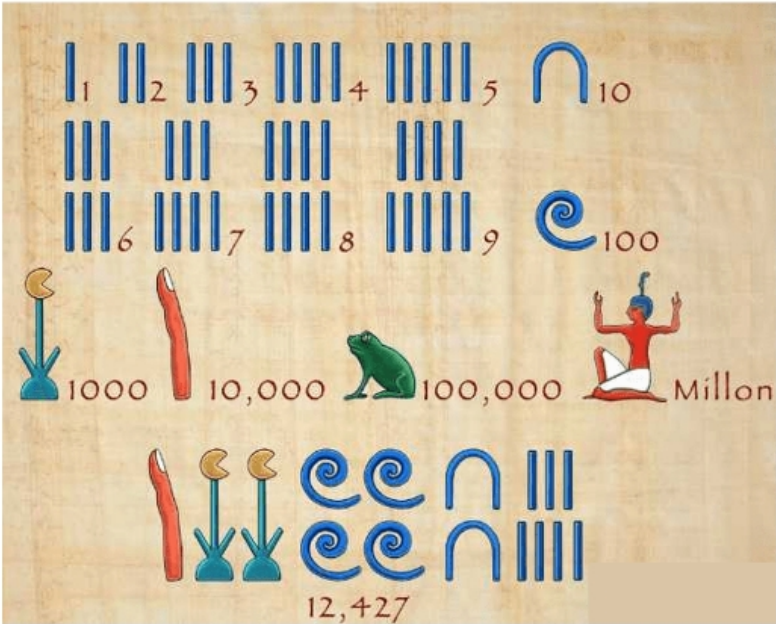
一、进制转换



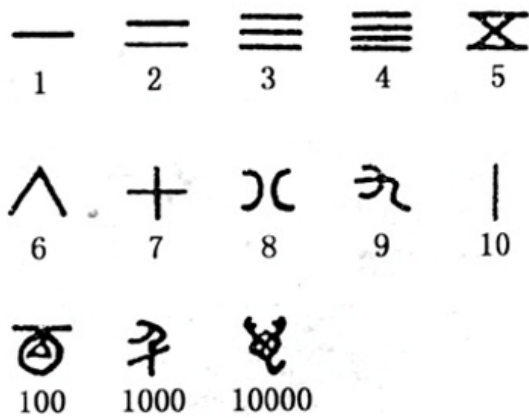
知识点精讲

1.1 什么是进制？

进制就是进位计数制，是人为定义的带进位的计数方法。



𐎀 1	𐎁 11	𐎂 21	𐎃 31	𐎄 41	𐎅 51
𐎆 2	𐎇 12	𐎈 22	𐎉 32	𐎊 42	𐎋 52
𐎌 3	𐎍 13	𐎎 23	𐎏 33	𐎐 43	𐎑 53
𐎒 4	𐎓 14	𐎔 24	𐎕 34	𐎖 44	𐎗 54
𐎘 5	𐎙 15	𐎚 25	𐎛 35	𐎜 45	𐎝 55
𐎞 6	𐎟 16	𐎠 26	𐎡 36	𐎢 46	𐎣 56
𐎤 7	𐎥 17	𐎦 27	𐎧 37	𐎨 47	𐎩 57
𐎪 8	𐎫 18	𐎬 28	𐎭 38	𐎮 48	𐎯 58
𐎱 9	𐎲 19	𐎳 29	𐎵 39	𐎷 49	𐎸 59
𐎹 10	𐎺 20	𐎻 30	𐎼 40	𐎽 50	



对于任何一种进制—X进制，每一位上的数做运算时都是逢X进一（借位时借一当X），各数位上的数字可以使用0,1,...X-1共 X 个数字来表示。（当阿拉伯数字0，1，2，3，4，5，6，7，8，9不够用时，也用引入固定的符号来表示数值，比如16进制用a表示10进制的10，b表示11，c表示12，d表示13，e表示14，f表示15）

1.2 常用进制

十进制	二进制	八进制	十六进制
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4

十进制	二进制	八进制	十六进制
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

```
1 //使用cout可以表示数字的不同进制
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     int a = 16;
7     cout << bitset<sizeof(a)*8>(a) << "\n"; //输出数字的八进制
8     cout << oct << a << "\n"; //输出数字的八进制
9     cout << dec << a << "\n"; //输出数字的十进制
10    cout << hex << a << "\n"; //输出数字的十六进制
11    return 0;
12 }
```

例题解析

2025年3月

第 8 题 下列程序的作用是 ()

```
1  int main() {  
2      int decimal = 25;  
3      cout << oct << decimal;  
4      return 0;  
5  }
```

A. 将十进制数转换成八进制数

B. 将八进制数转换成十进制数

C. 将二进制数转换成八进制数

D. 将八进制数转换成16进制数

第 9 题 下面程序是将十进制转十六进制，横线处应该填入的是（）

```
1  #include <iostream>  
2  using namespace std;  
3  int main() {  
4      int decimal = 255;  
5      _____  
6      return 0;  
7  }  
8  
9  A. cout << oct << decimal;  
10 B. cout << decimal << decimal;  
11 C. cout << hex << decimal;  
12 D. 不能正确执行
```

1.3 进制转换

在计数过程中采用进位的方法称为**进位计数制（进制）**，包括**数位**、**基数**和**位权**三个要素。

- **数位**：指数字符号在一个数中所处的位置。

- **基数**：指在某种进位计数制中数位上所能使用的数字符号的个数。（例如十进制的基数为10）
- **位权**：数制中某一位上的1所表示数值的大小（所处位置的价值）。（例如十进制的567，5的位权是100，6的位权是10，7的位权是1）

整数部分：“除 X 取余，逆序排列”

【1】将 X 作为除数，用十进制整数除以 X，可以得到一个商和余数；

【2】保留余数，用商继续除以 X，又得到一个新的商和余数；

.....

【3】如此反复进行，每次都保留余数，用商接着除以 N，直到商为 0 时为止。

【4】先得到的余数作为 X 进制数的低位数字，后得到的余数作为 X 进制数的高位数字，依次排列起来。

【例】将十进制数字 42 转为二进制

$$(42)_{10} = (101010)_2。$$

小数部分：“乘 X 取整，顺序排列”

【1】将十进制小数部分乘以 X，乘完后将乘积的整数部分取出；

【2】余下的小数部分继续乘以 X，再将新乘积的整数部分取出；

.....

【3】如此反复进行，每次都取出整数部分，用剩下的小数部分接着乘以 X，直到乘积中的小数部分为 0，或者达到所要求的精度为止。

【4】把取出的整数部分按顺序排列起来，先取出的整数作为 X 进制小数的高位数字，后取出的整数作为低位数字。

【例】将十进制数字 0.6875 转为二进制

$$(0.6875)_{10} = (0.1011)_2。$$

注：绝大部分浮点数无法用二进制精确表示

1.3.2. X 转十

方法：“按权展开求和”

- 【1】对于整数部分，从右往左看，第 i 位的位权等于 X^{i-1} ；
- 【2】对于小数部分，从左往右看，第 j 位的位权为 X^{-j} 。
- 【3】把每个数位上数值与对应的位权相乘后，将乘积相加求和。

【例】将八进制数字 53627 转换成十进制

$$(53627)_8 = 5 \times 8^4 + 3 \times 8^3 + 6 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 = (22423)_{10}$$

【例】将十六进制数字 9FA8C 转换成十进制

$$(9FA8C)_{16} = 9 \times 16^4 + 15 \times 16^3 + 10 \times 16^2 + 8 \times 16^1 + 12 \times 16^0 = 653964_{10}$$

【例】将二进制数字 1010.0.1101 转换成十进制

$$(1010.1101)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = (10.8125)_2$$

二转八：“三合一”

【整数部分】从低位向高位每三个二进制数分为一组，转换成一位八进制数字。高位不足三位前补零。

【小数部分】从高位向低位每三个二进制数分为一组，转换成一位八进制数字。低位不足三位后补零。

【例】将二进制整数 1110111100 转换为八进制

$$(1110111100)_2 = (1674)_8$$

二转十六：“四合一”

【整数部分】从低位向高位每四个二进制数分为一组，转换成一位十六进制数字。高位不足四位前补零。

【小数部分】从高位向低位每四个二进制数分为一组，转换成一位十六进制数字。低位不足四位后补零。

【例】将二进制整数 10 1101 0101 1100 转换为十六进制

$$(10110101011100)_2 = (2D5C)_{16}$$

八转二：“一分三”

将每一位八进制数字转换为三位二进制数字，把转换后的二进制数字拼在一起后，分别抹掉整数部分的高位零和小数部分的末尾零。

【例】将八进制整数 2743 转换为二进制

$$(2743)_8 = (10111100011)_2$$

十六转二：“一分四”

将每一位十六进制数字转换为四位二进制数字，把转换后的二进制数字拼在一起后，分别抹掉整数部分的高位零和小数部分的末尾零。

【例】将十六进制整数 A5D6 转换为二进制

$$(A5D6)_{16} = (1010010111010110)_2$$

注：八进制与十六进制间无法直接转换，需要借助二或十进制作为桥梁。

1. (L3-T7) 对于一个十进制数37，以下哪个是它的二进制表示()。

A. 10101 B. 100101 C. 101001 D. 1000101

2. (L3-T8) 下列关于十六进制的描述中，正确的是()。

A. 使用0-9和A-F表示 B. 使用0-9和A-E表示
C. 使用1-9和A-F表示 D. 使用1-9和A-E表示

3. (23-06-L3-T2) 二进制数11.01在十进制下是()。

A. 3.01 B. 3.05 C. 3.125 D. 3.25

真题复现

2025年3月

第 4 题 十进制数 111.111 的二进制表示可以是下面的 ()。

A. 1101111.0001110001

B. 1101110.1001110001

C. 1101111.1001110001

D. 1101111.0011110001

第 6 题 若将一个正整数化为二进制数，在此二进制数中，我们将数字 1 的个数是偶数的这类二进制数称为 A 类数，否则就称其为 B 类数。

例如：(13)₁₀=(1101)₂，其中 1 的个数为 3 则称此数为 B 类数；(10)₁₀=(1010)₂，其中 1 的个数为 2，称此数为 A 类数；

判断 (2025)₁₀ 化为二进制后，1 的个数为偶数个，因此 2025 为 A 类数

第 7 题 该段程序将 n 不停地除以 2，并输出此时的商和余数，直到 n=0 为止。

```
1  long long n;
2  cin >> n;
3  while(n != 0) {
4      cout << n/2 << ' ' << n%2 << '\n';
5      n /= 2;
6  }
```

第 8 题 两个 13 进制的数 A 和 B，在 10 进制下分别表示 10 和 11。(A+B)₁₃=(18)₁₃，也就是说 13 进制数 A 加上 13 进制数 B，和是 13 进制数 18。

第 9 题 k 进制，逢 k 进第二位，进百位，进千位；

第 10 题 CCF(十九进制)=21AC(十三进制)(不区分大小写)

(23-06-L3-T3) 已知大写字符 'A' 的 ASCII 编码的十六进制表示为 0x41，则字符 'F' 的 ASCII 编码的十六进制表示为()。

A. 46 B. 47 C. 48 D. 49

(23-06-L3-T8) 下列关于进制的叙述，不正确的是()。

- A. 正整数的二进制表示中只会出现0和1。
- B. 10不是2的整数次幂，所以十进制数无法转换为二进制数。
- C. 从二进制转换为8进制时，可以很方便地由低到高将每3位二进制位转换为对应的一位8进制位。
- D. 从二进制转换为16进制时，可以很方便地由低到高将每4位二进制转换为对应的一位16进制位。

(23-09-L3-T8) 下列关于进制的叙述，正确的是（ ）。

- A. 只有十进制和二进制能够用来表示小数，八进制和十六进制不可以。
- B. 常用的进制包括二进制、八进制、十进制、十六进制，其他进制在日常生活中很少使用。
- C. 对任意正整数，其二进制表示不会比它的十进制表示更短。
- D. 正整数的八进制表示中，每一位可能出现的最大数字是8。

2024-12 第 3 题 16进制数B2025转换成8进制数是（ ）

- A. 2620045
- B. 2004526
- C. 729125
- D. 2420045

2024-12 第 4 题 625.625变成二进制是（ ）。

- A. 1001110001.101
- B. 101.101
- C. 101.1001110001
- D. 1001110001.1101

2024-12 第 4 题 10 进制数 63，在转换成二进制的计算过程中，产生了如下的式子：

- 1 $63/2=31$ 余 1
- 2 $31/2=15$ 余 1
- 3 $15/2=7$ 余 1
- 4 $7/2=3$ 余 1
- 5 $3/2=1$ 余 1
- 6 $1/2=0$ 余 1

按照从前往后的顺序，获得 63 的二进制值是 111111

2024-6 第 3 题 8进制数3703转换成16进制数是 ()

A. 7C3

B. 7A3

C. 7B3

D. 7D3

2024-6 第 4 题 0.8125变成二进制是 ()。

A. 0.1110

B. 0.1 1111 1011 1101

C. 0.1 1111 1011 1100

D. 0.1101

2024-3 第 4 题 下列代码将十进制转化成八进制，则横线上应填入 ()

```
1  #include <iostream>
2  using namespace std;
3  void decimal2octal(int decimal) {
4      int oct_number[100];
5      int i = 0;
6      while (decimal > 0) {
7          _____ //在此处填入代码
```

```
8     }
9     for (int j = i - 1; j >= 0; j--) {
10         cout << oct_number[j];
11     }
12     cout << endl;
13 }
14
15 A. oct_number[i] = decimal % 8; decimal /= 8;
16 B. oct_number[i] = decimal / 8; decimal /= 8;
17 C. oct_number[i++] = decimal % 8; decimal /= 8;
18 D. oct_number[i++] = decimal / 8; decimal /= 8;
```

2024-3 第 5 题 二进制数101.11对应的十进制数是（ ）。

A. 6.5 B. 5.5 C. 5.75 D. 5.25

二、原码、反码、补码

知识点精讲

2.1 真值 (True Value)

- **定义：**数值的**实际数学值**，即人类理解的带有正负号的数值（如 `+5`、`-3.14`）。
- **特点：**
 - 包含符号（`+` / `-`）和绝对值（如 `-5` 的符号是 `-`，绝对值是 `5`）。
 - 与计算机存储无关，是理论上的精确值。
- **示例：**
 - 十进制真值：`+10`、`-255`
 - 二进制真值：`+1010`（对应 `+10`）、`-11111111`（对应 `-255`）

对于一个数字，计算机要使用一定的编码方式进行存储，原码、反码、补码是机器存储一个具体数字的3种具体编码方式。

2.2 机器数 (Machine Number)

- **定义**：数值在计算机中的**二进制编码形式**，是硬件实际存储和处理的表示。
- **特点**：
 - 必须使用固定位数（如 8 位、32 位）表示。
 - **符号和数值均被编码**（符号通常用最高位表示，如 0 正 1 负）。
 - 计算机无法直接存储 $+/ -$ 符号，需通过特定编码规则处理（如原码、反码、补码）。
- **常见编码方式**：
 - **原码**：最高位表示符号，其余位表示绝对值（如 -5 的 8 位原码：10000101）。
 - **反码**：负数符号位不变，数值位取反（如 -5 的反码：11111010）。
 - **补码**（最常用）：负数反码加 1（如 -5 的补码：11111011）。
- **示例**（8 位补码表示）：
 - +10 $\rightarrow$ 00001010
 - -10 $\rightarrow$ 11110110（补码）

2.3 原码

原码的表示方法是：最高位为符号位，0代表正数，1代表负数，非符号位用该数字绝对值的二进制表示。原码是人脑最容易理解和计算的表示方式。

例如当计算机字节长为8位时， $[+1]_{\text{原}} = 0000\ 0001$ ， $[-1]_{\text{原}} = 1000\ 0001$

第一位是符号位，只表示正负，8位二进制数原码的取值范围是： $[1111\ 1111, 0111\ 1111]$ ，即 $[-127, 127]$

2024-9 第 1 题 +1和-1的原码进行 $1+(-1)$ 计算的结果是-2 。（)

2024-6 第 3 题 一般默认64位计算机系统中整型变量（int） 还是32位，则整数能够表示的数据范围是()。

A. $[0, 2^{32}]$

B. $[0, 2^{64}]$

C. $[-2^{31}, 2^{31} - 1]$

$$D. [-2^{63}, 2^{63} - 1]$$

2.4 反码

反码的表示方法是：正数的反码是其本身；负数的反码是在其原码的基础上，符号位不变，其余位取反。

例如 $[+1] = [0000\ 0001]$ 原 = $[0000\ 0001]$ 反, $[-1] = [1000\ 0001]$ 原 = $[1111\ 1110]$ 反；

如果一个反码表示的是负数，人脑无法直观的看出来它的数值。通常要将其转换成原码再计算。

8位二进制数反码的取值范围时： $[1000\ 0000, 0111\ 1111]$ ，即 $[-127, 127]$

2.5 补码

补码的表示方法是：正数的补码就是其本身；负数的补码是在其原码的基础上，符号位不变，其余各位取反，最后+1。(也即在反码的基础上+1)

$[+1] = [0000\ 0001]$ 原 = $[0000\ 0001]$ 反 = $[0000\ 0001]$ 补

$[-1] = [1000\ 0001]$ 原 = $[1111\ 1110]$ 反 = $[1111\ 1111]$ 补

对于负数，补码表示方式也是人脑无法直观看出其数值的。也需要转换成原码再计算其数值。

8位二进制数补码的取值范围是： $[1000\ 0000, 0111\ 1111]$ ，即 $[-128, 127]$

1、所以正数的原码、反码、补码都相同。

2、计算机中负数都是以补码形式存在的。

2025-3 第 5 题 在C++中，补码的主要作用是（）

A. 提高浮点数的精度

B. 简化整数的加减法运算

C. 增加整数的表示范围

D. 优化内存分配

2025-3 第 6 题 在C++中，一个8位有符号整数（使用补码表示）的范围是（）

A. -128 到 127

B. -127 到 128

C. -256 到 255

D. 0 到 255

2024-6 第 2 题 整数-6的16位补码可用十六进制表示为FFFA。

2024-6 第 3 题 补码的优点是可以将减法运算转化为加法运算，从而简化计算机的硬件设计。

2.6 读一读 存在多种编码的原因

原码表示法是最容易理解的，最高位0或1表示正负，数值位表示具体的大小，为什么要引入反码和补码出现呢？

存在即合理！在计算机中都是一系列0和1组成的二进制参与基础运算（加减乘除），依据数学运算法则定律：减一个正数等于加上一个负数，而乘法是加法的累积，除法是减法的累积，所以在计算机中只要有加法就够了。这样就无需计算机无需辨别符号位从而让计算机的基础电路设计变得更加简单。

①用**原码**计算十进制表达式： $1 - 1 = 0$

$$1 - 1 = 1 + (-1) = [0000\ 0001]_{\text{原}} + [1000\ 0001]_{\text{原}} = [1000\ 0010]_{\text{原}} = -2$$

如果用原码表示，让符号位也参与计算，显然对于减法来说结果是错误的。为了解决原码做减法的问题，出现了**反码**。

②用**反码**算十进制的表达式： $1 - 1 = 0$

$$1 - 1 = 1 + (-1) = [0000\ 0001]_{\text{原}} + [1000\ 0001]_{\text{原}} = [0000\ 0001]_{\text{反}} + [1111\ 1110]_{\text{反}} = [1111\ 1111]_{\text{反}} = [1000\ 0000]_{\text{原}} = -0$$

用**反码**计算十进制表达式： $5 - 3 = 2$

$5 - 3 = 5 + (-3) = [0000\ 0101]_{\text{原}} + [1000\ 0011]_{\text{原}} = [0000\ 0101]_{\text{反}} + [1111\ 1100]_{\text{反}} = [1\ 0000\ 0001]_{\text{反}}$ (因为进位溢出截取最高位1) $= [0000\ 0001]_{\text{反}} = [0000\ 0001]_{\text{原}} = 1$

发现用反码计算减法，结果的真值部分是正确的。而唯一的问题其实就出现在"0"这个特殊的数值上。对于原码和反码而言存在 +0 和 -0 两个特殊数值，显然0带符号是没有任何意义的，但是会存在[0000 0000]原和[1000 0000]原两个编码表示0。**补码**的出现，解决了 0 的符号问题以及 0 的两个编码问题。

③用**补码**计算十进制的表达式： $1 - 1 = 0$

$1 - 1 = 1 + (-1) = [0000\ 0001]_{\text{原}} + [1000\ 0001]_{\text{原}} = [0000\ 0001]_{\text{补}} + [1111\ 1111]_{\text{补}} = [1\ 0000\ 0000]_{\text{补}}$ (因为进位溢出截取最高位1) $= [0000\ 0000]_{\text{补}} = [0000\ 0000]_{\text{原}} = 0$

用**补码**计算十进制表达式： $5 - 3 = 2$

$5 - 3 = 5 + (-3) = [0000\ 0101]_{\text{原}} + [1000\ 0011]_{\text{原}} = [0000\ 0101]_{\text{反}} + [1111\ 1100]_{\text{反}} = [0000\ 0101]_{\text{补}} + [1111\ 1101]_{\text{补}} = [1\ 0000\ 0010]_{\text{补}}$ (因为进位溢出截取最高位1) $= [0000\ 0010]_{\text{补}} = [0000\ 0010]_{\text{原}} = 2$

用**补码**计算十进制表达式： $(-1) + (-127) = -128$

$(-1) + (-127) = [1000\ 0001]_{\text{原}} + [1111\ 1111]_{\text{原}} = [1111\ 1111]_{\text{补}} + [1000\ 0001]_{\text{补}} = [1000\ 0000]_{\text{补}}$

在补码运算中，[1000 0000]补其实就是-128，但实际上将以前的 -0 表示为-128的补码形式，所以-128并没有原码和反码表示。(对-128的补码表示[1000 0000]补，算出来的原码是[0000 0000]原，是不正确的)

使用补码，不仅仅修复了0的符号以及存在两个编码的问题，而且还能够多表示一个最低数。这就是为什么8位二进制，使用原码或反码表示的范围为[-127, +127]，使用补码表示的范围为[-128, 127]。

同理对于常用到的有符号的32位int类型，可以表示范围是： $[-2^{31}, 2^{31} - 1]$ 。

例题解析

1. (L3-T1-14462) 下列关于负数的原码、反码、补码的描述中，正确的是()。

- A. 原码和反码互为按位取反（符号位除外），补码为反码加1
- B. 原码和反码互为按位取反（符号位除外），补码为原码加1
- C. 反码和补码互为按位取反（符号位除外），原码为反码加1
- D. 补码和原码互为按位取反（符号位除外），反码为补码加1

2024-12 第 1 题 下列二进制表示的十进制数值分别是 ()

$[10000011]_{\text{原}} = ()$ $[10000011]_{\text{补}} = ()$

A. -125, -3

B. -3, -125

C. -3, -3

D. -125, -125

2024-9 第 2 题 关于计算机中的编码, 下列说法中正确的是 ()

A. 机器数的形式值和真值是一致的

B. 原码就是符号位加上真值

C. 机器数是带符号的

D. $[-1]_{\text{补}} = [1000\ 0001]_{\text{原}} = [1111\ 1111]_{\text{反}}$

真题复现

2025-3 第 7 题 在C++中, 以下代码的输出是什么()

```
1  int a = -5;  
2  unsigned int b = a;  
3  cout << b;
```

A. -5

B. 5

C. 4294967291

D. 编译错误

2024-12 第 2 题 关于计算机中的编码，下列说法中错误的是（）

- A. 对于无符号数，原码就是真值
- B. 正数的反码是其本身
- C. 负数的反码和补码是不一样的
- D. 负数的反码，在其原码的基础上, 各个位取反

2024-12 第 6 题 补码1111 1101进行运算1111 1101>>1以后得到的结果是（）

- A. 1111 1100
- B. -2
- C. 1111 1101
- D. 1111 1010

三、位运算

知识点精讲

计算机中的数在内存中都是以二进制形式进行存储的，位运算是将每个二进制位作为布尔值，分别对一个或两个二进制数中对应的二进制位做**布尔运算**。

注意：在手动进行位运算计算时，需要将数转换成二进制的表示形式，再进行计算。如 $3 \& 5$ ，先写成二进制形式的 0011 和 0101，右边最低位对齐后再分别对相应位进行位运算。

3.1 位运算规则介绍

3.2 位运算常用性质

【简单规则】

(1) a 与自身之间的操作

- 1 $a \& a = a;$
- 2 $a \mid a = a;$
- 3 $a \wedge a = 0;$

(2) a 与 0 之间的操作

- 1 $a \& 0 = 0;$
- 2 $a \mid 0 = a;$
- 3 $a \wedge 0 = a;$

(3) 还原计算

- 1 $a \mid (a \& b) = a;$
- 2 $a \& (a \mid b) = a;$

【基础性质】

“清零取反要用与，某位置一可用或。若要取反和交换，轻轻松松用异或”

(4) 判断奇偶

- 1 // 通过 $\& 1$ 取出最后一个二进制位以达到模 2 的效果，位运算效率更高
- 2 $\text{if}(a \& 1 == 1)$ // 奇数
- 3 $\text{if}(a \& 1 == 0)$ // 偶数
- 4 // 同 $(a \% 2)$

(5) 比较两哥整数是否相等

- 1 $a \wedge b$ // 如果结果为 0 则为相等，反之不等

(6) 正整数乘或除以 2^x

```
1  a  = a << x; //乘以2^x;  
2  a  = a >> x; //除以2^x;
```

(7) 交换两个整数

```
1  a ^= b;  
2  b ^= a;  
3  a ^= b;
```

【高阶延展】

(8) 取出 a 中倒数第 i 个二进制位上的数值

```
1  a & (1 << (i - 1)); 或 (a >> (i - 1)) & 1
```

(9) 计算 a 的相反数

```
1  a = ~a + 1; //数字取反后等于其相反数-1
```

(10) 保留 a 在二进制位中最后一个 1

```
1  a &= (-a); //获取最低位位权，常用在lowbit()函数中
```

(11) 删除 a 二进制序列中最后一个值为 1 的位置

```
1  a &= (a - 1); //如果结果为0，则说明a是2的指数
```

(12) 将 a 的倒数第 i 位二进制位设为 1

```
1  a |= 1 << (i - 1);
```

(13) 将 a 的倒数第 i 位二进制位设为 0

```
1 a &= ~(1 << (i - 1));
```

(14) 保留 a 二进制序列中最后的 i 位，其余补 0

```
1 a &= ((1 << i) - 1);
```

(15) 将 a 二进制序列中最后 i 位全部置为 0

```
1 a &= ~(1 << i) - 1;
```

(16) 求 a 取模 2^x 后的结果

$a = a \& (2^x - 1);$ 同 $a \% 2^x$

3.3 运算符优先级

例题解析

1. (L3-T2) 一个int类型的值乘以8，等价于以下哪个位运算()。

A. 左移3位 B. 右移3位 C. 左移8位 D. 右移8位

2. (L3-T3) 以下哪个属于C++语言中的位运算符()。

A. + B. - C. * D. &

3. (L3-T11) 如果a和b均为int类型的变量，下列表达式能够正确判断“a等于0且b等于0”的是()。

A. $((\sim a) \&\& (\sim b))$ B. $((a \& b) == 0)$ C. $((a | b) == 0)$ D. $((a \wedge b) == 0)$

4. (L3-T12) 如果 a 为 int 类型的变量，下列哪个表达式可以正确求出满足“大于等于a且是4的倍数”的整数中最小的()。

A. $(a \& (\sim 3))$ B. $(a / 4 * 4)$ C. $((a - 1) | 3) + 1$ D. $(a << 2)$

5. (L3-T15) 在下列代码的横线处填写()，可以使得输出是 17 11 。

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a = 11,b = 17;
5      a = _____; //在此处填入代码
6      b = a ^ b;
7      a = a ^ b;
8      cout << a << " " << b << endl;
9      return 0;
10 }

```

A. $a + b$ B. $a - b$ C. $a \wedge b$ D. $a \& b$

6. (23-06-L3-T4) 以下哪个不是C++语言中的运算符()。

A. & B. && C. * D. **

7. (23-06-L3-T10) 一个int类型的值，做以下哪个操作，一定会变回原来的值()。

A. 左移3位，再右移3位。

B. 右移3位，再左移3位。

C. 按位或7，再按位与-8。

D. 按位异或7，再按位异或7。

8. (23-06-L3-T11) 如果a和b均为int类型的变量，下列表达式能正确判断“a等于b”的是()。

A. $((a / b) == 1)$ B. $((a \& b) == a)$ C. $((a \wedge b) == 0)$ D. $((a | b) == b)$

9. (23-06-L3-T12) 如果a为int类型的变量，下列哪个表达式可以正确求出满足“小于等于a且是4的倍数”的整数中最大的()。

A. $(a \& (\sim 3))$ B. $((a << 2) >> 2)$ C. $(a \wedge 3)$ D. $((a - 1) | 3) + 1$

10. (23-06-L3-T13) 在下列代码的横线处填写()，可以使得输出是 24 12 。

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a = 12,b = 24;
5      _____; // 在此处填入代码
6      a = a ^ b;
7      b = a ^ b;
8      cout << a << " " << b << endl;
9      return 0;
10 }

```

A. $a = a \wedge b$ B. $b = a \wedge b$ C. $a = a + b$ D. $b = a + b$

11. (23-09-L3-T4) 以下哪个不是C++语言中的运算符? ()

A. ~ B. ~~ C. < D. <<

12. (23-09-L3-T10) 如果 a 是 int 类型的变量, 下列哪个表达式的值一定为 true? ()

A. $a + 1000 - 1000 == a$ B. $a * 2 / 2 == a$ C. $(a \& 1) == 1$ D. $(a | 1) == a + 1$

13. (23-09-L3-T11) 如果 a 和 b 均为 int 类型的变量, 下列表达式不能正确判断“a 等于 b”的是 ()。

A. $((a \geq b) \&\& (a \leq b))$ B. $((a >> 1) == (b >> 1))$ C. $((a + b) == (a + a))$ D. $((a \wedge b) == 0)$

14. (23-06-L4-T10) 如果a为int类型的变量, 且a的值为6, 则执行 `a &= 3;` 之后, a的值会是()。

A. 3 B. 9 C. 2 D. 7

15. (23-09-L4-T11) 如果 a 为 int 类型的变量, 且 a 的值为6, 则执行 `a = ~a;` 之后, a 的值会是 ()。

A. -6 B. 6 C. -7 D. 7

四、数据结构之一维数组

概念

可以用来存储一系列同种类型的数据，并且这些数据在内存中是连续存储的。（线性结构、顺序结构）

定义

1. 数据类型 数组名 [数组长度];
2. 数据类型 数组名[数组长度] = {值1, 值2...};
3. 数据类型 数组名[] = {值1, 值2...};

声明及初始化

```
1  int a[1005];           //如果定义在主函数之上，数组a所有元素的值赋为0
2  int b[5]={1,2,3,4.5}; //b[0]、b[1]、b[2]、b[3]、b[4]的值分别赋为1,2,3,4,5
3  int c[5]={1,2,3};      //c[0]、c[1]、c[2]、c[3]、c[4]的值分别赋为1,2,3,0,0
4  int d[5]={0};          //将数组d中5个元素的值全赋为0
5  int e[]={1,2,3};       //e[0]、e[1]、e[2]的值分别赋为1,2,3，且将数组的长度定义为3
```

【注意事项】

- 1.局部数组默认值为野值，全局数组默认值为0；
- 2.数组长度可以使是一个常量或包含运算符的表达式；

```
1  int n = 10;
2  int a[n]; //此定义方式并不适合所有的编译器，只有支持 C99 标准的编译器才支持这种定义方式，否则在编译时会报错，
```

- 3.大数组须定义为全局数组，在局部定义大数组会爆栈；

使用

数组名[下标]

数组下标从0的位置开始，下标最大值为数组长度减1。

例如 `a[0]`、`b[1]`、`c[b[i]]`、`d['L']`

只能逐个引用数组元素的值，而不能一次引用整个数组中全部元素的值。

【注意事项】

1.数组名是常量，不可以进行赋值操作；整型数组输出数组元素时，输出数组名得到的是首元素地址而不是批量元素。

2.使用数组时下标**不能越界**（上界和下界），否则会造成内存的混乱，但**不会导致编译错误**；

计算占用空间

计算数组占用内存总空间的大小，主要有两种方式

①数组相当于有多个变量组成，可以通过单个数组元素所占用的空间乘以数组长度

②也可以使用sizeof（数组名）得到整个数组占用的内存大小

```
1  int array[10];
2  printf("array占用的内存大小为=%d\n",sizeof(array));    // i占用的内存大小为=40
3  printf("int占用的内存大小为=%d\n",sizeof(int) * 10);    // int占用的内存大小为=4
```

函数（选讲）

```
1  int arr[4]={1,3,2,4};
```

排序：sort(起始地址，结束地址，排序规则);（地址遵循“左闭右开”的原则）

```
1  sort(arr,arr+4);//对a[0~a[3]四个元素按升序排序
```

填充：fill(起始地址，结束地址，值);

```
1  fill(a,a + 4, 5);//将a[0]~a[3]之间的元素全部填充为数字5
```

初始化：memset(数组, 值, 空间代销);

- 1 `memset(a, 0, sizeof(a));` //将数组a元素全部初始化为0
- 2 注意：`memset`函数是按照字节来赋值的！

反转：`reverse`(起始地址，结束地址);

- 1 `reverse(a, a + 4);` //将a[0]~a[3]之间的元素全部翻转

数组应用

(1) 求和、平均、极差

【例】好声音大赛

在今年的好声音歌唱比赛中，每名选手会由 n 名裁判进行打分（百分制）。请你按“掐头去尾”的原则计算选手的平均分与极差（最大值与最小值之差），保留三位小数。

```
1  #include <stdio>
2  int main(){
3      int n;
4      int x, sum = 0, MIN = 101, MAX = -1;
5      scanf("%d", &n);
6      for (int i = 1; i <= n; i++){
7          scanf("%d", &x);
8          if(x > MAX) MAX = x;
9          if(x < MIN) MIN = x;
10         sum += x;
11     }
12     printf("%.3lf %d", (sum - MAX - MIN) * 1.0 / (n-2), MAX - MIN);
13     return 0;
14 }
```

(2) 查找或统计

【例】找重复数

在一次数学考试结束后，班级 n 位 ($2 \leq n \leq 100$) 同学中有两位同学考了了相同的分数（百分制），请你找到此分数并输出。

```

1  #include <cstdio>
2  int main(){
3      int n, a[105], cnt[100] = {};
4      cin >> n;
5      for (int i = 1; i <= n; i++) {
6          cin >> a[i];
7          cnt[a[i]]++;
8      }//方法1: 枚举查找
9      for (int i = 1; i <= n; i++) {
10         for (int j = i + 1; j <= n; j++) {
11             if (a[i] == a[j]) {
12                 cout << a[i];
13                 break;
14             }
15         }
16     }
17     //方法2: 计数原理
18     for (int i = 0; i <= 100; i++) {
19         if (cnt[i] > 1){
20             cout << i;
21             break;
22         }
23     }
24
25     return 0;
26 }

```

(3) 计数或标记

【例】明明的随机数

明明用计算机生成了N个1到1000之间的随机整数 ($N \leq 100$)，对于其中重复的数字只保留一个，把其余相同的数去掉，然后再把这些数从小到大排序。请你协助明明完成“去重”与“排序”的工作。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3

```

```

4  int main(){
5      int n, x, k, cnt[1001] = {};
6      cin >> n;
7      for(int i = 1; i <= n; i++){
8          cin >> x;
9          if(cnt[x] == 0){
10             k++;
11         }
12         cnt[x]++; // cnt[x] = 1;
13     }
14     cout << k << '\n';
15     for(int i = 1; i <= 1000; i++){
16         if(cnt[i])    cout << i << ' ';
17     }
18     return 0;
19 }

```

(4) 递推

【例】走台阶

一个楼梯有 n ($1 \leq n \leq 30$) 级，小李同学从下往上走，每一步可以跨一步，也可以跨两步。请问走到第 n 级台阶有多少种走法？

```

1  #include <cstdio>
2  int main(){
3      int n, a[31];
4      cin >> n;
5      a[1] = 1, a[2] = 2;
6      for (int i = 3; i <= n; i++) {
7          a[i] = a[i - 1] + a[i - 2];
8      }
9      cout << a[n];
10     return 0;
11 }

```

【真题训练】

1. (L3-T4) 若有以下代码，则数组arr的长度是(C)。

```
int arr[] = {1,2,3,4,5};
```

A. 3 B. 4 C. 5 D. 6

2. (L3-T5) 在C++语言中，可以定义一个一维整型数组的是(A)。

A. int array[5] B. int array[] C. int[5] array D. int[] array

3. (23-06-L3-T6) 一个数组定义为double array[3];，则这个数组占用内存的大小为(A)。

A. 24 B. 12 C. 6 D. 3

4. (23-06-L3-T7) 以下数组定义，符合C++语言语法的是(B)。

A. int a[]; B. int b['3']; C. int c[3.0]; D. int [3] d;

5. (23-06-L3-T14) 在下列代码的横线处填写(D)，可以使得输出是 2 。

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int array[5] = {3,7,5,2,4};
5      int min = 0;
6      for (int i = 0; i < 5; i++)
7          if (_____) // 在此处填入代码
8              min = array[i];
9          cout << min << endl;
10     return 0;
11 }
```

A. min > array[i] B. min < array[i] C. min = array[i] D. 以上均不对

6. (23-06-L3-T15-14832) 在下列代码的横线处填写(B)，可以使得输出不是 31 。——电子讲义选题

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int array[5] = {1,2,4,8,16};
5      int res = 0;
6      for (int i = 0; i < 5; i++)
7          _____ // 在此处填入代码
8      cout << res << endl;
9      return 0;
10 }

```

A. $res = res + array[i]$

B. $res = res \& array[i]$

C. $res = res | array[i]$

D. $res = res \wedge array[i]$

7. (23-09-L3-T5-16002) 如果数组定义为 `long long array[] = {3, 5, 7, 2};` , 则数组 `array` 占用的字节数为 (A) 。 ——电子讲义选题

A. 32 B. 16 C. 8 D. 4

8. (23-09-L3-T6) 一个数组定义为 `double array[3];` , 则可合理访问这个数组的元素的下标最大为 (A) 。

A. 2 B. 3 C. 23 D. 24

9. (23-09-L3-T7) 以下数组定义, 符合C++语言语法的是 (B) 。

A. `double a[];` B. `double b[] = {1, 2.0, '3'};` C. `double c[3.0];` D. `double[] d = new double[3];`

10. (23-09-L3-T9-16006) 下列关于C++语言中数组的叙述, 不正确的是 (C) 。 ——电子讲义选题

A. 可以定义 0 个元素的数组。

B. 不能定义 -1 个元素的数组。

C. 数组下标越界访问会产生编译错误。

D. 程序运行时发生数组下标的越界访问，程序依然可能正常结束。

11. (23-09-L3-T14) 题 在下列代码的横线处填写 (D)，可以使得输出是“120”。

A. `res += array[i]` B. `res *= array[i]` C. `res = array[i]` D. 以上均不对。

12. (23-09-L3-T15) 在下列代码的输出是 (B)。

A. 15 B. 28 C. 45 D. 55

13. (L4-14) 在下列代码的横线处填写(B)，可以使得输出是 21。

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a[5];
5      a[0] = 1;
6      for (int i = 1; i < 5; i++)
7          a[i] = a[i - 1] * 2;
8      int sum = 0;
9      for (int i = 0; i < 5; _____) // 在此处填入代码
10         sum += a[i];
11     cout << sum << endl;
12     return 0;
13 }
```

A. `i++` B. `i += 2` C. `i += 3` D. `i |= 2`

五、字符串及函数

概念

字符串是一个接一个字符的一组字符连在一起（类似于用一根冰糖葫芦串起了很多山楂球），所以字符串的基本组成元素是字符。

储存

存储字符串比如"Jcak"时，有两种不同的方式，一种是字符型**数组**，另一种是**string**类。

```
1 char a[5] = "jcak";
```

在字符串中使用标识符 `'\0'` 作为结束标志，结束符 `'\0'` 也需要占用一个存储位，所以声明数组时长度至少为5。等同于：

```
1 char a[5] = {'j', 'a', 'c', 'k', '\0'};
```

【注意事项】

- 1.由 n 个字符数组成的字符串在内存中应占（n+1）个字节。
2. `'0'` 代表字符0，对应ASCII码值为48，`'\0'` 代表空字符(转义字符)，对应ASCII码值为0。

输入与输出

方法1：可以使用 `%s` 整体输入输出（不保存空白符，如空格、回车、换行符等）

方法2：利用循环结合下标逐次操作

```
1 #include <stdio>
2 int main(){
3     char a[10];
4     scanf("%s", a); //无需加&
5     printf("%s", a);
6     return 0;
7 }
```

函数

(1) 字符转换

判断字符ch是否为小写字符，如果是转为大写字符

```
1 //方法1:
2 if (ch >= 'a' && ch <= 'z') 或 if (ch >= 97 && ch <= 122)
3     ch = ch - 'a' + 'A';//或ch -= 32;
4 //方法2: <cctype>头文件下
5 if (islower(ch)) {
6     toupper(ch);
7 }
```

判断一个字符ch是否为大写字符，如果是转为小写字符

```
1 //方法1:
2 if (ch >= 'A' && ch <= 'Z') 或 if (ch >= 65 && ch <= 90)
3     ch = ch - 'A' + 'a';//或ch += 32;
4 //方法2: <cctype>头文件下
5 if (isupper(ch)) {
6     tolower(ch);
7 }
```

判断一个是否为数字字符

```
1 //方法1:
2 if (ch >= '0' && ch <= '9') 或 if (ch >= 48 && ch <= 57)
3 //方法2: <cctype>头文件下
4 if (isdigit(a[i]))
```

(2) 获取长度

使用 `strlen()` 可以计算字符串中有效字符个数（不包括 `'\0'`）


```
1 char a[5] = "jcak";
2 cout << strlen(a); //5
```

(3) 替换

通过 `strcpy(a, b)` 来把复制字符型数组b内的信息，并替换掉数组a保存的字符串。（返回值是数组a首字符的指针）

```
1 char a[5] = "jcak", b[5] = "rose";
2 strcpy(a, b);
3 cout << a; //rose
```

(4) 拼接

使用 `strcat(a, b)` 可以把字符型数组b保存的信息连接在原数组a保存的之后。（返回值是数组a首字符的指针）

```
1 char a[10] = "jcak", b[5] = "rose";
2 strcat(a, b);
3 cout << a; //jackrose
```

(5) 比较大小

使用 `strcmp(a, b)` 按照字典序逐位比较对应位置的字符大小，如果字符相同继续向后进行比较，若不同返回比较后的结果。如果 $a < b$ ，比较结果为-1；如果 $a = b$ ，比较结果为0；如果 $a > b$ ，比较结果为1；

```
1 char a[10] = "jcak", b[5] = "rose";
2 cout << strcmp(a, b) << endl; // -1
3 cout << strcmp(b, a) << endl; // 1
```

【真题训练】

1. (L3-T9) 下列哪个是 C++语言中用于获取字符串长度的函数(D)。

A. length() B. len() C. getLength() D. strlen()

2. (L3-T14-14483) 在下列代码的横线处填写(D), 可以使得输出是 1357 ,不会有多余字符。——电子讲义选题

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  int main() {
5      char str[] = "1234567";
6      for (_____)//在此处填入代码
7          cout << str[i];
8      return 0;
9  }
```

A. int i = 0; i < strlen(str); i++

B. int i = 0; str[i] != '\0'; i++

C. int i = 1; i <= 7; i += 2

D. int i = 0; i <= 6; i += 2

3. (23-06-L3-T5-14822) 如果字符串定义为char str[]="Hello";, 则字符数组str的长度为(C)。——电子讲义选题

A. 0 B. 5 C. 6 D.7

4. (23-06-L3-T9) 下列关于C++语言中数组的叙述, 不正确的是(C)。

A. 数组必须先定义后使用。

B. 数组的所有元素在内存中是连续存放的。

C. 除了字符数组, 在定义数组时 “[]” 内必须有常数。

D. 不能对数组赋值, 但可以对数组的每个基础类型的元素赋值。

5. (23-09-L3-T3) 已知大写字母 'A' 的ASCII编码的十六进制表示为 0x41 , 则字母 'L' 的ASCII编码的十六进制表示为 (C)。

A. 4A B. 4B C. 4C D. 52

6. (23-09-L3-T12) 如果 a 为 `char` 类型的变量，下列哪个表达式可以正确判断“ a 是大写字母”？ (D)

A. $a - 'A' \leq 26$ B. $'A' \leq a \leq 'Z'$ C. $'A' \leq 'a' \leq 'Z'$ D. $('A' \leq a) \ \&\& \ (a \leq 'Z')$

六、编程题

知识点精讲

模拟法

现实生活中有些问题难以找到公式或规律来求解，只能按照一定的步骤不停的模拟下去，最后才能得到答案。这样的问题用计算机求解十分合适，只要能让计算机模拟人在解决此问题时的行为即可。这种求解问题的思想，称为模拟。

真题复现

2025年3月 词频统计

在文本处理中，统计单词出现的频率是一个常见的任务。现在，给定 n ($1 \leq n \leq 100$) 个单词，你需要找出其中出现次数最多的单词。在本题中，忽略单词中字母的大小写（即 `Apple`、`apple`、`APPLE`、`aPPlE` 等均视为同一个单词）。请你编写一个程序，输入 n 个单词，输出其中出现次数最多的单词。

2024年6月 移位

小杨学习了加密技术移位，所有大写字母都向后按照一个固定数目进行偏移。偏移过程会将字母表视作首尾相接的环，例如，当偏移量是3的时候，大写字母 `A` 会替换成 `D`，大写字母 `Z` 会替换成 `C`，总体来看，大写字母表 `ABCDEFGHIJKLMNOPQRSTUVWXYZ` 会被替换成 `DEFGHIJKLMNOPQRSTUVWXYZABC`。注：当偏移量是26的倍数时，每个大写字母经过偏移后会恰好回到原来的位置，即大写字母表 `ABCDEFGHIJKLMNOPQRSTUVWXYZ` 经过偏移后会保持不变。

2024年9月 回文拼接

一个字符串是回文串，当且仅当该字符串从前往后读和从后往前读是一样的，例如，`aabaa`和`ccddcc`都是回文串，但 `abcd`不是。

小杨有 $n(1 \leq n \leq 10)$ 个仅包含小写字母的字符串(长度不超过100)，他想请你编写程序判断每个字符串是否由两个长度至少为 2 的回文串前后拼 接而成。

2024年12月 数字替换

小杨有一个包含 个数字的序列 A，他想将其中大于 k的数字都替换为序列的最大值，将 其中小于k 的数字都替换为序列的最小值，请你帮他计算出替换后的序列。

2024年12月 打印数字

小杨为数字 0, 1, 2和 3 设计了一款表示形式，每个数字占用了 的网格。数字 , , 和 的表示形式如下：

```
1  ..... ****. ....
2  .***. ****. ****. ****.
3  .***. ****. ....
4  .***. ****. .**** ****.
5  ..... ****. ....
```

小杨想请你将给定的数字 n 转换为对应的表示形式。

枚举法

逐一列举问题所涉及的所有情形，并根据问题提出的条件检验排查出是问题的解。（也称为列举法、穷举法，是暴力策略的具体体现）

- （1）枚举算法的思路简单，正确性容易证明，方便程序编写和调试
- （2）当问题的规模变大的时候，运算量过大，执行速度越慢，解题效率不高

- （1）确定枚举对象；
- （2）确定枚举范围和判断条件；

(4) 逐一筛选出问题的解。

常用语法：循环+判断语句。

由于枚举法对要所有可能的情况进行筛选，不重复不遗漏地进行检验，通过牺牲时间来换取答案的全面性。为此优化的主要方向是：

(1) 减少状态总数，从而减少计算两；

(2) 将原问题化为更小的问题，根据问题的性质进行截枝；

【经典例题】求极值问题、百元买百鸡、阿斯特朗数

真题复现

2025年3月

小 A 有一个整数 $x(0 \leq x < 2025)$ ，他想找到最小的正整数 y 使得下式成立：

$$(x \text{ and } y) + (x \text{ or } y) = 2025$$

其中 and 表示二进制按位与运算，or 表示二进制按位或运算。如果不存在满足条件的，则输出 -1。

2024年6月

寻找倍数

小杨有一个包含 n 个正整数的序列，他想知道是否存在 i 使得 a_i 是序列中所有数的倍数。共 t 组询问。

2024年9月

平衡序列

小杨有一个包含 n 个正整数的序列 a ，他认为一个序列是平衡的当且仅当存在一个正整数 i 使得序列第 1 个到第 i 个数字的总和等于第 $i+1$ 个到第 n 个数字的总和。小杨想请你判断序列是否是平衡的。共有 t 组询问。

对于全部数据，保证有 $1 \leq t \leq 100, 1 \leq n, a_i \leq 10000$

读一读 算法

基本概念

算法（Algorithm）是指解题方案的准确而完整的描述。在计算机科学领域中算法几乎无处不在，在各种计算机系统的实现中，算法的设计往往处于核心的位置。

算法特征

特性	描述
有穷性	有穷性是指必须能在执行有限个步骤之后结束，并且每一步都可以在有限的时间内完成
确切性	每一个步骤必须具有确切的定义，是无二义性的
输入性	一个算法有0个或多个输入（所谓0个输入是指算法本身就嵌入了输入）
输出性	一个算法必须具有一个或多个输出，以反映算法对输入数据加工后的结果
可行性	任何操作都是可以分解为基本的可执行的步骤

标准

标准	介绍
正确性	正确性是指在合理的数据输入下，能在有限的时间内得出正确的结果
可读性	算法主要是为了人的阅读与交流，其次才是让计算机执行，因此算法应该易于人的理解
健壮性	算法应当具备检查错误和对错误进行适当处理的能力
效率	指算法执行时所需计算机资源的多少，包括 运行时间 和 存储空间 两方面的要求（时间复杂度、空间复杂度）

描述方式

自然语言描述

自然语言法是指用文字叙述的形式描述集合的方法。自然语言来描述算法的优点是更便于对算法的理解和阅读，缺点是不够严谨易产生歧义。当使用自然语言描述比较复杂的结构算法时，就不够直观清晰了。

流程图

使用特定**图形符号**加上**文字**说明表示**具体流程**和**算法思路**的一种框图，通俗地讲就是“**流程**”+“**图**”。除了说明程序的流程顺序外，着重于说明程序的逻辑性。

伪代码描述

伪码语言介于高级程序设计语言和自然语言之间，它忽略高级程序设计语言中一些严格的语法规则与描述细节，因此它比程序设计语言更容易描述和被人理解，而且比自然语言或算法框图更接近程序设计语言。

使用特定的可以直接在计算机上执行的程序描述算法。优点是不用转换直接可以编译执行，缺点是需要对特定的程序设计语言比较理解。（所谓“程序”是指对所要解决问题的各个对象和处理规则的描述，或者说是数据结构和算法的描述，因此有人说 数据结构+算法=程序）

算法设计方法主要有分治策略、动态规划、贪心算法、回溯法、搜索、概率算法等。

【真题训练】

1. (L3-T6) 枚举算法的主要特点是()。

A. 以空间换时间 B. 逐个尝试所有可能的解 C. 动态规划 D. 贪心策略

2. (L3-T10) 通常用下列哪种方式来描述算法()。

A. 汇编语言 B. 伪代码 C. SQL D. CSS